

assignment01

Introduction to Computer Science M – Spring 2025 - Assignment A1

version 1.0

Rules:

- **Submission Deadline: April 20 at 23:59.** Late submissions will incur the following penalties:
 - **April 21:** 10-point deduction
 - **April 22:** 20-point deduction
 - **April 23:** 30-point deduction

Submissions will not be accepted after April 23.

- The assignment should be completed in groups of two students (already assigned in Moodle).
- Students will be randomly selected to provide an oral explanation of their solution to the exercises.
- Questions regarding the exercise statements are welcome during office hours. However, professor will not assess your solution.
- There is no guarantee that questions submitted via email will receive a response.
- The code must be implemented in C. Submitted code should compile (**no compilation errors** if compiled with gcc). This is the minimum requirement. Note that graders and professors will assess the correctness and quality of the submitted code. **Compilation warnings** will be translated to deduction in the grades.
- **Important:** Your programs should only use C features covered in weeks 1-5 of the course. In addition, the following restrictions apply:
 - Do not use **break** or **continue** statements.
 - The program must contain only one **return** statement.

If a solution violates the above restriction, **it will receive a grade of 0.**

- You should submit a separate file containing the source code for each exercise (ensure the file names match the requirements for each task). Alternatively, you may submit a single zip file containing the code for all exercises.

Plagiarism and Academic Misconduct:

In cases of copying or plagiarism, exams will be placed under review. If plagiarism is confirmed, the case will be reported to the GTIT Discipline Committee. **This includes submitting solutions found online or generated by AI tools.** All group members will be held accountable for any submission that violates this policy.

Exercise 1 : Digits of a number [15 points]

Task 1: Reversing a number.

Write a C program `reverse.c` that asks the user for an integer `n` and then prints all of the `n`'s digits backwards. For example, for the number 123, the program should print 3, 2 and 1. Utilize a do-while.

```
***
Number : 12345
Reverse: 54321
***
```

Hint: utilize the modulo operation (%) to get the last digit of a number.

Task 2: Quantity of digits of a number.

Write a program `quantity.c` to compute the number of digits in the given natural number and determine its greatest digit. Below are some expected outputs.

- Example 1:
Number : 12321
Digits : 5
Greatest: 3
- Example 2:
Number : 879
Digits : 3
Greatest: 9
- Example 3:

```
Number : 0
Digits : 1
Greatest: 0
```

Task 3: Repunit.

Write a program `repunit.c` to determine if a given number is a **repunit** number. A number is considered a repunit if it consists solely of the digit 1, such as 1, 11, 111, 1111, and so on.

- Example 1:

```
Number : 11111
Repunit : Yes
```

- Example 2:

```
Number : 123
Repunit : No
```

- Example 3:

```
Number : 0
Repunit : No
```

Exercise 2 : Generalization of Fibonacci and Pell Sequences [40 points]

Assume a fixed constant c and define the sequence $x_0, x_1, \dots, x_n$ with

$$x_0 = 0$$

$$x_1 = 1$$

$$x_{n+2} = x_n + c * x_{n+1} \text{ for } n \geq 0$$

Note that:

- When $c == 1$, it produces the **Fibonacci sequence**.
- When $c == 2$, it generates the **Pell sequence**.

Task 1: Generating Triplets (`triplets.c`)

Write a C program `triplets.c` that prompts the user to enter:

- A constant c .
- A quantity q .

The program should then print the first q triplets (a, b, c) , where a , b , and c are three consecutive numbers in the sequence for the given c . Your program should ensure that the user enter positive numbers.

Constraints: In addition to the general constraints, the solution may only use `while` or `do-while` loops for iteration. If the solution contains `for` loops, the grade will be 0.

Example:

```
Enter the number of triplets to display: -10
Enter the number of triplets to display: 10
Enter the constant c: 0
Enter the constant c: 1
(0, 1, 1)
(1, 1, 2)
(1, 2, 3)
(2, 3, 5)
(3, 5, 8)
(5, 8, 13)
(8, 13, 21)
(13, 21, 34)
(21, 34, 55)
(34, 55, 89)

Insert the numbers of tuples to be shown: 5
Insert the constant c: 2
(0, 1, 2)
(1, 2, 5)
(2, 5, 12)
(5, 12, 29)
(12, 29, 70)
```

Task 2: Finding Triplet for a perimeter (`greater.c`)

The *perimeter* of a triplet is the sum of its three components, i.e., the perimeter of (a, b, c) is $a + b + c$. Write a C program, `greater.c`, that prompts the user for a perimeter p and a constant c and then finds and prints the first triplet in the sequence corresponding to c that has perimeter **greater than** p . The program should display the triplet and the perimeter.

Constraints: In addition to the general constraints, the solution may only use `while` or `do-while` loops for iteration. If the solution contains `for` loops, the grade will be 0.

Examples

```
Enter the perimeter: 10
Enter the constant c: 1
The tuple (3, 5, 8) in the series for c = 1 has perimeter 16 (> 10)

Enter the perimeter: 30
Enter the constant c: 3
The tuple (3, 10, 33) in the series for c = 3 has perimeter 46 (> 30)
```

Task 3: Finding the Closest Triplet (`closest.c`)

Write a C program `closest.c` that asks the user for a perimeter `p` and a constant `c` and finds and prints the triplet in the sequence corresponding to `c` that has the **closest** perimeter to `p`. The program should inform if the closest triplet have a perimeter that is **smaller than, equal to or greater than** than `p`.

Hint: The triplet with the closest perimeter is the last one with perimeter smaller or equal to; or the first one with largest perimeter.

Constraints: In addition to the general constraints, the solution may only use `while` or `do-while` loops for iteration. If the solution contains `for` loops, the grade will be 0.

Examples

```
Enter the perimeter: 10
Enter the constant c: 1
The closest tuple (2, 3, 5) for the constant c = 1 has perimeter 10 (equal to 10)

Enter the perimeter: 11
Enter the constant c: 1
The closest tuple (2, 3, 5) for the constant c = 1 has perimeter 10 (smaller than 11)

Enter the perimeter: 15
Enter the constant c: 1
The closest tuple (3, 5, 8) for the constant c = 1 has perimeter 16 (greater than 15)
```

Task 4: Finding a Constant (`constant.c`)

Write a C program `constant.c` that prompts the user for a perimeter `p` and determines the constant `c` that yields a triplet where:

- The first component is **not** 0, and
- The perimeter is the **closest** to `p` among the perimeters of all triples (for any `c`) with a perimeter less than or equal to `p`. **Triples with perimeter greater than `p` should not be considered.**

The program should print the constant c and the perimeter of the closest triplet.

Hint: The maximum possible value of c that can be a solution is related to the given perimeter p .

Examples

```
Enter the perimeter: 19
the best constant c = 2 has a tuple with perimeter 19
```

```
Enter the perimeter: 15
the best constant c = 3 has a tuple with perimeter 14
```

```
Enter the perimeter: 2
There is no such constant
```

[Bonus] If your program outputs the triplet with the closest perimeter, a bonus of 3 points will be added to your assignment grade.

Exercise 3: Maze (45 points)

We will simulate a simple game consisting of a maze with one treasure, where the user can move around to find the treasure.

The maze is a square grid with 5 rows and 5 columns, represented using Cartesian coordinates. The bottom-left corner is at coordinates ($x = 0$, $y = 0$), and the top-right corner is at ($x = 4$, $y = 4$). The maze contains a treasure at a specific position, and the user also starts inside the maze. This simple maze representation will help visualize the game's grid, where U represents the user at position (0, 0) and T represents the treasure located at (3, 4).

	column (x)	

4	. . . T .	
3		
2		Line (y)
1		
0	U 	

	0 1 2 3 4	

You can represent the position of the treasure using two variables, e.g., $x_treasure = 3$ and $y_treasure = 4$. Similarly, for the user's position, you can use $x_user = 0$ and $y_user = 0$.

Game Description

The game proceeds as follows:

- The user selects a direction: **u** for up, **d** for down, **l** for left, and **r** for right.
 - The user moves to a new position based on the selected direction. For instance, if the user is at (0, 0) and moves **r**, the position changes to (1, 0). If the user moves **u**, the position becomes (0, 1). If the user chooses a direction that would move them outside the maze (e.g., left or down when in position (0,0)), they will remain in the same position.
 - The game ends when the user reaches the treasure.
-

Task 1

Write a C program `maze.c` that:

- Prompts the user for the coordinates of the treasure, i.e., `x_treasure` and `y_treasure`.
- Initially positions the user at the coordinate `x_user = 0, y_user = 0`.
- Continuously asks the user to enter a direction (**u, d, l, r**), and moves the user to the corresponding coordinate.
- The game continues until the user reaches the treasure.

You may assume that the user will enter correct information. There is no need to check for invalid position of the treasure or directions other than (**u, d, l, r**).

Task 2

Write a C program `maze_err.c` that modifies the solution from Task 1 to:

- Check for invalid treasure coordinates (e.g., coordinates outside the 5x5 grid) and prompt the user again if the coordinates are invalid.
 - Ensure the user can only enter valid directions (**u, d, l, r**). If the user enters an invalid direction, ask them to enter a valid one.
-

Task 3

Write a C program `maze_info.c` that builds upon the solution from Task 2, with the following additions:

- At the end of the game, the program displays the total number of moves the user made to reach the treasure.

- It also shows how many attempts the user made to move outside the maze (i.e., when they tried to move beyond the grid boundaries).
-

Task 4

Write a C program `maze_limited.c` that modifies the solution from Task 3 to:

- End the game if the user makes more than 3 invalid attempts to move outside the maze (i.e., tries to move outside the grid boundaries).
- The program should inform the user that they have lost the game if they exceed 3 invalid moves, or inform them they have won otherwise.